Post-Compromise Security with Persistent Randomness Reservations

Research manuscript — 7 November 2026

Abstract

A persistent messaging protocol can reserve randomness so that interrupted cryptographic operations can resume with their original inputs. This separates the creation of secret material from the execution, publication, and confirmation of the operation that uses it. We study the resulting exposure semantics in a hybrid post-quantum messaging construction. Our record model assigns immutable origins to reserved entropy and follows those origins through pending writes, aggregate images, acquisition, and later wrapping-key disclosure. A record-interface normalization lemma relates alias-aware storage to an original-record experiment while preserving acquisition and readability. For the protocol's multi-epoch symbolic model, a jointly checked secrecy theorem establishes recovery under fresh target reservations, unexposed target authentication and entropy, and a precise record-acquisition condition. Native executions demonstrate the necessity of provenance-sensitive reasoning: a captured reservation reconstructs later traffic, a reservation captured during asynchronous confirmation exposes a successor beyond the smaller confirmation counter, and a sibling-session write preserves the original secret source. We organize these results with executable SPQR and whole-KEM reference corpora, and formulate the common computational experiment needed to connect concrete protected records to joint root, confirmation, traffic, and acknowledgement security. The analysis makes the origin and acquisition history of retained secret material explicit in the recovery statement.

1 Introduction

Recovery after compromise depends on the origin of the secrets that a protocol adds to its state. Ratcheting protocols renew communication keys by combining existing state with new contributions, and their security analyses explain when later keys regain secrecy after earlier state has been exposed [CGCD+20, DJK+25]. In a durable implementation, however, the random choices for a contribution can enter persistent storage before the cryptographic operation runs. A restart may then execute that operation for the first time using randomness that was already present in a compromised record. The distinction between reserving a secret and using it is therefore part of the recovery argument.

This separation arises naturally when interrupted operations must resume with the same output. Q-Periapt Continuity, the construction studied in this paper, records a protected randomness reservation before computing a rekey offer or response. The reservation fixes the original operation so that a crash, retry, or delayed storage acknowledgement preserves its meaning. This provides a useful durability contract: the resumed computation belongs to the original exchange, and an already prepared public message can be retransmitted exactly. It also gives secret material a lifetime that begins before publication, execution, and confirmation. An analysis that uses any of these later events as the birth of fresh entropy can consequently assign secrecy to material that an adversary already possesses.

A motivating execution. Consider an authenticated rekey from root R_{e-1} to root R_e. An endpoint first stores the randomness reservation for a KEM key-generation operation. Before the key pair is computed, an adversary acquires the protected reservation, its matching wrapping key, and the previous root. The process then crashes. After restart, the honest endpoint resumes the reserved operation, publishes its offer, and completes the exchange and its confirmation messages. The adversary can reconstruct the same private key from the captured reservation and obtain the new KEM contribution from the later public response. Combining that contribution with R_{e-1} yields the same descendant root and traffic keys. Every protocol message in this execution can be honestly authenticated.

We realize this execution at the actual persistent key-generation boundary and at the analogous encapsulation boundary. In each case, the honest peers finish the exchange, reopen their journals, and exchange six newly generated application messages that the predictor successfully decrypts. The predictor uses the captured inputs and the public transcript; its subsequent derivation interface has no journal or future-private-state input. These executions establish a concrete separation between post-restart computation and post-compromise entropy.

Asynchronous confirmation exposes a second distinction. One endpoint may have committed an epoch and reserved a contribution for its successor while the other still awaits a receipt. Our delayed-receipt execution captures precisely such a state: the lagging peer has confirmed epoch zero, yet the captured reservation exposes epoch-two traffic. The record that matters is the one containing the contribution's ancestor, rather than the smaller of the two confirmed counters. A third execution shows that writing another session into an aggregate device image preserves the target reservation's original origin. A later image revision and a newly sampled wrapping nonce change the storage representation, while the retained KEM randomness remains the same.

The question studied here. Which reservation and record-acquisition conditions suffice for a later epoch's messages to remain secret when operations are resumed, confirmations are asynchronous, and roots are inherited across epochs? We answer this question for a multi-epoch symbolic abstraction of Continuity and construct native executions that isolate the role of the conditions. The organizing object is an *original reservation*: a sampled contribution associated with its owner, operation, session, and epoch. Retrying that operation or copying its record preserves this origin. Freshness is stated in terms of reservation and disclosure events, and the theorem derives message secrecy from those events. We also distinguish the creation of a session-record version from the later creation or resealing of an aggregate image that carries it. An acquisition records which original versions were obtained, even when their readability changes later because a wrapping key is disclosed.

This question fits naturally with the set-based account of exposure developed for sparse continuous key agreement (SCKA) [ADJ$^+$25]. That account already distinguishes sending and receiving epochs, future key material, and retained out-of-order message keys. Our focus is the persistent state beneath those interfaces: when the randomness for a future contribution became recoverable, which original operation a resumed call executes, and which records an adversary acquired. We use the existing set-based perspective to organize reference experiments and make these additional state relationships explicit.

Contributions. The paper develops three connected results.

1. **Exposure accounting and a mechanized multi-epoch recovery theorem.** We give an origin-preserving record interface and a normalization lemma relating immutable record aliases to their originals, with the rendering interface explicit in both experiments. A conditional computational accounting proposition separates record-interface

error, source attribution, component security, and the joint root/traffic residual. We prove symbolic message secrecy for a target epoch whose original key-generation and encapsulation reservations both follow the compromise cut, whose actual authentication keys and target-epoch reserved entropy remain undisclosed, and for which no acquired target-session record was saved after the cut. The model supports inherited roots, alternating proposers, asynchronous confirmation, and unbounded epochs. It admits exposure of old roots and wrapping keys and tracks actual key identities across their reuse. The main theorem and all 24 reusable dependencies were checked together in an independent Tamarin replay [Q-P26]. A separately checked two-round ground execution supplies a concrete instance satisfying the complete positive witness formula.

2. **Constructive separations at the persistence and confirmation boundaries.** Native process-kill experiments reconstruct the future KEM contribution from each of the two reserved-operation boundaries and decrypt messages after honest confirmation and restart. Further executions establish exposure beyond the lagging peer's next confirmed epoch and preservation of an old reservation through a sibling session's aggregate write. A symbolic delayed-receipt ground trace isolates the same freshness condition: the generation reservation precedes the cut while the encapsulation reservation follows it. These witnesses explain why the theorem refers to original reservations and acquired record contents.
3. **Executable comparisons that preserve the meaning of exposure.** We retain a pinned upstream SPQR implementation and an authenticated whole-KEM control at three fixed intervals. Across seven communication schedules, these form 28 configuration–schedule combinations and 336 endpoint-snapshot cases. Independent derivation programs reconstruct available message keys and compare them with actual sender outputs. Results retain direction, epoch, and message index, together with the contribution that enabled each derivation. This supplies a reproducible basis for comparing retained secrets and communication opportunities while keeping authentication and persistence interfaces explicit.

Scope of the security statement. The mechanized recovery theorem establishes symbolic message secrecy under the stated reservation and record-acquisition conditions. The native results establish the corresponding concrete disclosure executions. The computational accounting proposition identifies the common-experiment bounds needed to connect protected records, a replaced KEM contribution, and joint root and traffic uses. Its concrete instantiation is under development. Throughout, crash recovery means resuming retained, bound operations under the specified authentication and storage model. This makes the recovery claim precise in relation to the state-loss and conversation-level conditions studied in prior work [CJN23, CMN25].

The resulting distinction is operationally useful: confirmation records protocol progress, while reservation provenance and disclosure history determine the premises of recovery. Keeping these quantities separate permits exact replay and asynchronous progress to coexist with a precise secrecy argument.

2 Related work

Our comparison separates the security of a ratcheting construction, the exposure represented by its state, and the interpretation of that state in a durable implementation. This places the reservation problem alongside existing cryptographic and systems results without identifying a new storage write, a completed exchange, and a recovered secret as the same event.

Ratcheting and exposure of future messages. Cohn-Gordon et al. analyze Signal's X3DH and Double Ratchet core as a multi-stage authenticated key-exchange protocol [CGCD+20]. Their model represents the branching structure of ratchet stages and specifies admissible key disclosures for a challenged stage. This establishes an important precedent for deriving a precise cryptographic object from a deployed state machine. The object studied here includes a contribution's recoverable randomness before its KEM computation, together with the record versions that retain it. Our witnesses connect these earlier objects to later roots and application messages.

Auerbach et al. introduce SCKA and vulnerable epoch and message sets [ADJ+25]. Definition 3.2 includes current chains, already produced future-epoch keys, and pending out-of-order message keys. Theorem 3.3 gives a secure-messaging construction from secure SCKA, an appropriate KDF, and AEAD. Their Section 1.2 explicitly identifies state loss and session handling as a separate setting. We use their set-based perspective and investigate how original reservations and persistent-record acquisition determine exposure at that representation boundary. In particular, our delayed-receipt witness is compared with the state of both directions and their retained contributions, rather than assigned a universal bound from a confirmed-epoch counter.

The timing of secret generation is itself an established design consideration. Section 8.5 of Signal's Double Ratchet specification discusses deferring ratchet key generation until it is needed for sending, thereby shortening a key's lifetime [PMS25]. Our contribution concerns the operational meaning of that lifetime when a durable reservation fixes the key before the generation call runs. An implementation can defer computation while retaining the original coins. The relevant freshness event is then the reservation, and its identity must survive retries and subsequent image writes.

Session handling and state-loss resilience. Cremers, Jacomme, and Naska distinguish individual ratcheting sessions from the conversation assembled by a messaging application [CJN23]. Their Signal experiments and Tamarin model show how session selection and retry behavior permit a clone to remain active, and they analyze two mechanisms for stronger conversation guarantees. This is directly relevant to the scope of our disclosure interfaces: a session label identifies a protocol instance, whereas shared authentication or wrapping keys can connect several instances. The Continuity model follows actual key identities, and the aggregate-copy execution examines a concrete storage relationship between two sessions.

Cremers, Medinger, and Naska subsequently prove incompatibilities between full conversation PCS and specified state-loss resilience requirements [CMN25]. Their Definitions 1 and 2 require continued communication after dynamic or static state loss; the distinction includes losing evolving communication state and losing persistent identity state. Our crash/resume setting retains a bound journal and resumes its original operations under explicit authentication assumptions. The recovery theorem states exactly which reservation, identity, and record-acquisition histories are admissible. Thus, the retained state and independent authentication in our claim identify the particular recovery setting being studied. The delayed-receipt and aggregate-copy witnesses explain how that setting behaves during honest asynchronous progress.

Robustness on unreliable networks. Fischlin, Günther, and Janson parameterize channel correctness by supported delivery histories and develop corresponding robustness notions [FGJ24]. Their analysis of QUIC and DTLS 1.3 accounts for loss, reordering, replay policies, and packet-number reconstruction. In particular, adversarial traffic should preserve acceptance of supported honest ciphertexts. This provides a useful methodological parallel: legitimate network behavior belongs inside a security model's operational semantics. We

retain delayed receipts and asymmetric cutover when reasoning about reservation provenance. Delivery admissibility and secret freshness are separate predicates on the same execution; synchronizing both peers before allowing the next reservation would change the execution that our witnesses and theorem analyze.

Post-quantum ratchets and communication comparisons. Dodis et al.'s Triple Ratchet combines classical and post-quantum continuous key agreement, using erasure coding to distribute the larger post-quantum messages [DJK+25]. Its Section 4 develops the hybrid construction and security argument. Remark 4.3 already accounts for exposure of next-epoch material sampled while its transmission is in progress. We therefore compare original-reservation persistence with an existing analysis of retained future material. Our SPQR and whole-KEM experiments report the scheduling and retention consequences of particular implementations; their authentication mechanisms, control opportunities, and persistence interfaces remain explicit experimental parameters.

The reference corpus separates the pinned upstream SPQR code from the experimental whole-KEM construction. The latter is a comparison control with fixed local-send intervals. It contributes an executable full-message exchange for the same schedule corpus. A case reports the concrete message keys derived from one endpoint snapshot and the public transcript; its `not_derived` label records the output of that derivation program. This convention preserves the distinction between an observed exposure set and the security statement supplied by a cryptographic theorem. It also allows reordered traffic to retain its direction and message index instead of being compressed into a single recovery time.

Mechanized ratchet analyses. Cheval, Jacomme, and Richards give an automated analysis of the Double Ratchet that includes out-of-order messages, key-compromise scenarios, and PCS [CJR26]. Their work also identifies and analyzes fixes for forward-secrecy defects in the specification and implementation. These results make detailed ratchet state and compromise interfaces central comparison points for our symbolic analysis.

The CCS 2026 program includes Bhargavan et al.'s analysis of SPQR and its implementation [BBC+26]. The authors' published account describes ProVerif models used during protocol design and hax/F* verification of the Rust core's absence of panics and selected correctness properties [Bha25]. Our comparison with this work concerns the state represented by an exposure and the provenance preserved by durable replay. The present contribution combines a checked recovery condition with concrete reservation-disclosure executions and an explicit normalization of record aliases.

Hérouard et al. mechanize a computational PCS proof for an asymmetric ratchet in Squirrel [HJKL26]. Their analysis covers two parties and unbounded ratchet steps with a fixed, non-adaptive compromise scenario. The proof distinguishes non-deduction of intermediate roots from real-or-random security of derived keys, and develops rules for reasoning about these properties in the random-oracle model. This distinction informs our computational interface: each intermediate-root claim is attached to the view in which it is used, and message-key indistinguishability is a separate conclusion. A convention that samples ideal names in advance is also distinct from giving an adversary a persistent token from which actual operation coins can be recovered; the disclosure interface determines which values enter its view.

Key derivation with an exposed predecessor. Continuity's root extraction invokes HMAC with the previous root in the key position and the new contribution in the message position. RFC 5869 specifies this argument order for HKDF-Extract [KE10]. When the predecessor root is exposed, the relevant secret therefore occurs in HMAC's message input.

Event	Effect on the target operation	Provenance fact
Reserve	Draw KEM entropy and seal its replay token	Creates an original source
Persist target	Retain the original pending-write bytes	Creates an acquisition opportunity
Apply or retry	Install or replay the retained target	Preserves the source
Sibling write	Re-encode the aggregate journal with a new outer nonce	Preserves unchanged target sources
Confirm	Advance the relevant endpoint's protocol state	Records protocol progress

Table 1: The events tracked by the analysis. Several events may occur in one symbolic transition; the native execution exposes their separate boundaries.

Backendal et al. characterize the input-domain feasibility conditions for HMAC's swapped PRF use and give security bounds under compression-function assumptions [BBGS23]. These results supply a precise reference for that extraction step. Our computational accounting additionally identifies the record interface, source attribution, and joint root/traffic experiment to which an instantiation must apply.

Across these comparisons, the specific object added by the present study is the provenance of retained operation randomness through reservation, record acquisition, replay, and asynchronous confirmation. The normalization result preserves that provenance at the record interface, the symbolic theorem gives a sufficient recovery condition, and the native witnesses show why the condition refers to the original events.

3 Construction and the motivating execution

Q-Periapt Continuity combines ML-KEM-768 and X25519 in its ContextBound hybrid KEM and authenticates protocol controls with an ML-DSA-65/P-256 identity. Our analysis follows the persistent session and rekey machinery: an original operation first reserves its entropy, places the resulting token in a retained plan, and later computes or replays the corresponding KEM operation. An exact retry uses the same reservation. The protocol alternates the proposing endpoint across epochs and carries separate send, receive, and confirmed-epoch state.

3.1 Reservations and persistent representations

A generation reservation determines the future hybrid key pair. An encapsulation reservation, together with the authenticated offer, determines the corresponding ciphertext and contribution. The implementation seals these reservations under the host wrapping key. It persists the selected target in a pending write before installing that target as the next journal image. Applying or reopening that write preserves its original contents.

A journal image aggregates state from multiple sessions. Updating a sibling session advances the global image revision and samples a new outer encryption nonce, while the target session's retained reservation can remain unchanged. Consequently, the journal revision, outer nonce, local operation identifier, and original entropy reservation represent distinct identities. We preserve these distinctions in the model and relate them to the native execution boundaries.

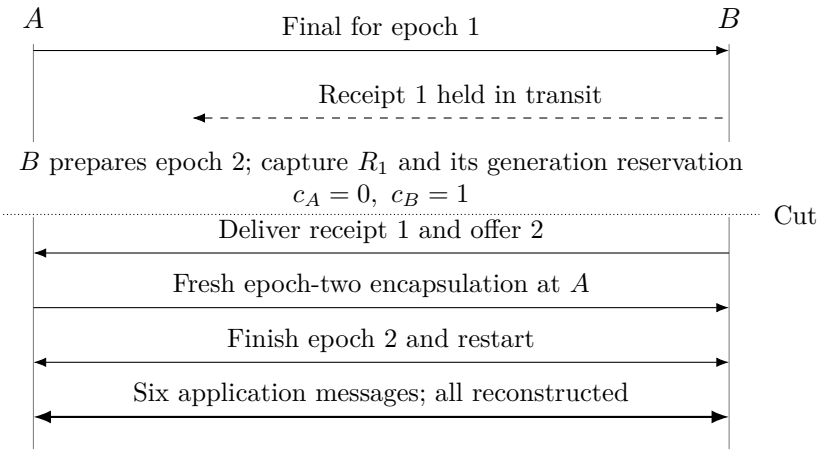


Figure 1: A retained native disclosure execution. The smaller confirmed counter is zero at capture, while the captured reservation enables epoch-two traffic recovery after the public exchange. The dotted Cut orders the exposure and the subsequent fresh encapsulation; it does not refresh the earlier generation reservation.

3.2 Root and traffic dependencies

Write R_{e-1} for the preceding root, S_e for the hybrid contribution, C_e for the public response core, and O_e, P_e for the complete signed offer and response. With the construction's canonical encodings and domain-separated transcript hashes, its dependency equations are

$$R_e = \mathrm{HKDF}_{\mathrm{SHA256}}(R_{e-1}, S_e, d_R \| H(d_C, C_e), 32), \tag{1}$$

$$T_e = \mathrm{HKDF}_{\mathrm{SHA256}}(sid, R_e, d_T \| ctx \| e \| H(d_{\mathrm{tr}}, O_e \| P_e), 128). \tag{2}$$

Here HKDF arguments denote salt, input key material, information, and output length in bytes. T_e supplies two directional traffic chains and two acknowledgement keys. The actual domain strings and encodings are pinned by the source locations in Appendix A.

These equations describe a concrete derivation path. A captured generation reservation and matching wrapping key reconstruct the private key; a subsequent public ciphertext then yields S_e. A captured encapsulation reservation yields the same contribution using its retained offer. Combining S_e with a disclosed R_{e-1} and the public control wires reproduces R_e and the traffic material. Honest admission checks can still reject an operation inside the SDK while an adversary holding these inputs performs the underlying computation independently.

3.3 Asynchronous confirmation and a later exposed epoch

Consider the native execution in Figure 1. Endpoint B commits the first receipt, advances locally, and prepares an epoch-two offer before A receives that receipt. At the capture point their local observations are

$$A : (c, s, r, p) = (0, 1, 0, 1), \qquad B : (c, s, r, p) = (1, 1, 1, 2),$$

where c, s, r are confirmed, send, and receive epochs, and p is the pending target epoch. The snapshot of B contains R_1 and the original generation reservation for epoch two. After delivery resumes, those captured inputs and the public exchange reconstruct epoch-two traffic.

The resulting counterexample targets the proposed bound $\min(c_A, c_B) + 1$: its value is one, yet epoch-two traffic is exposed. The protocol's asymmetric progress is legitimate. Recovery reasoning must therefore follow the retained inputs and their descendants rather than identify the two endpoints' confirmation counters.

4 Reservation Origins, Persistent Records, and Recovery

An interrupted cryptographic operation can retain its random tape before it has produced a key or a ciphertext. Its later execution and confirmation therefore have to be related to the original reservation. We make this relationship explicit at two levels: the origin of a cryptographic operation, and the origin of a saved session-state version. A third identifier describes the physical image that carries that version. This distinction lets us account for a target record copied by another session's write, an unapplied pending image, and a ciphertext acquired before its wrapping key.

The model is an interface model for an asynchronous protocol machine $\mathcal{P}$. The adversary schedules public messages, process loss, honest recovery, and the disclosure operations specified below. Application deliveries are private to the honest applications. Capture grants read access to the stated assets; it does not grant an additional operation for replacing the authoritative journal. An implementation with adversarial rollback or forking requires those transitions in its storage machine. The protocol, its cryptographic primitives, and its public outputs are parameters of the record-normalization lemma. Fix finite execution, record-size, and query budgets; the wrapper's bookkeeping is polynomial in their total representation size.

4.1 Three origins and their event order

Definition 4.1 (Reservation origin)**.** A reservation origin is a fresh invocation index o, together with

$$\big(\mathsf{kind}_o, \mathsf{owner}_o, s_o, e_o, B_o, \mathsf{res}(o)\big).$$

Here kind_o distinguishes generation, encapsulation, and any other reserved operation; s_o, e_o identify its session and epoch; B_o is the full canonical binding, including the admitted policy and operation inputs; and $\mathsf{res}(o)$ is the event at which its random tape ρ_o is drawn. Execution or exact replay of o consumes the same ρ_o and the same binding. A new draw receives a new origin even if a failed attempt's external operation identifier is reused.

The ideal source in this model draws independent tapes of the prescribed lengths. Explicit entropy-disclosure queries reveal designated tapes. Independence is a distributional assumption about this source, rather than a consequence of a later API call. The generation and encapsulation budgets G, C count original draws, including draws in attempts that subsequently fail. Exact retries consume no new slot. Independently sampled operations retain distinct origins even if some public outputs happen to coincide.

A record version v has an immutable payload X_v and semantic metadata

$$\big(\mathsf{subject}(v), \mathsf{role}(v), \mathsf{epoch}(v), \mathsf{phase}(v)\big).$$

The subject identifies the session whose state is represented, rather than the session that most recently wrote the enclosing journal. A version can refer to several reservation origins and to roots or traffic state derived from earlier origins. Preparing a version allocates its handle. The event $\mathsf{Saved}(v)$ occurs when that version first enters the durable acquisition surface, and its time is denoted $\mathsf{save}(v)$. In particular, an intact target in a durable pending-write intent is saved before the corresponding image is applied. A storage operation may have made this transition even when its caller receives an unknown-outcome error.

A physical object h has a separate creation or sealing index. It may carry one version, several sessions' versions, an exact earlier image, or a pending target. An unchanged version keeps its v and $\mathsf{save}(v)$ when it is copied, archived, applied from a pending intent, or included in a newly sealed aggregate. A new semantic session-state version receives a new v even when

some of its fields are unchanged. These rules distinguish the original reservation from each session-state update that retains it.

The following transition rules specify the provenance discipline.

1. *Reserve and execute.* Reserve creates o at the actual draw. Execution uses its retained binding and tape. Unknown-outcome reconciliation keeps that origin; a definite new attempt allocates another.
2. *Prepare and retain.* Preparing v fixes X_v and its provenance. Retaining the first intact image or pending target containing v emits $\mathsf{Saved}(v)$. An attempt that retains no such object has no successful acquisition event.
3. *Copy, apply, and reseal.* An exact copy preserves the complete object and its sealing origin. Resealing creates a new sealing invocation and its actual new wrapper randomness, while preserving the referenced record versions. Applying an existing pending target copies its retained bytes.
4. *Crash and resume.* Process loss removes volatile references. Recovery reconstructs the retained handles and their original provenance. It does not turn a retained reservation into a new draw.
5. *Confirm and delete.* Confirmation changes protocol state and may authorize particular host-side deletions. Neither action changes a reservation's origin or removes an adversary's previously acquired copy.

For an original reservation actually retained in v, causality requires

$$\mathsf{res}(o) \;\leq\; \mathsf{save}(v). \tag{3}$$

A later physical object's creation time gives an additional timestamp, not a replacement for either side of (3). The interval between drawing ρ_o and first retaining it remains explicit. The symbolic reservation-and-save transition can represent that interval only through a separately justified source correspondence.

4.2 Acquisition, delayed readability, and key identity

Wrapping-key handles resolve to identities of actual key material. Two session labels that designate the same key resolve to the same identity κ. Disclosure through either label therefore enables every acquired wrapper protected by κ. A rotation creates a different identity when it creates different key material. Signing-key identities are treated in the same way across sessions and roles.

For each intact physical object h, its construction records a finite set

$$\mathsf{paths}(h) = \{(v, \gamma)\},$$

where γ is the set of wrapping-key identities on a recoverable serialization path to X_v. Multiple paths to one version are permitted. An unprotected record has $\gamma = \varnothing$. Nesting a wrapper adds its key identity; aggregation unions its members' paths. An exact ciphertext copy keeps its paths. Resealing decoded records reconstructs the paths under the new wrapper and retains the same v values. Thus resealing decoded plaintext and wrapping an old ciphertext are distinct operations.

Only intact, recoverably embedded record payloads create these paths. A public hash or MAC of a private record is a public computation of that record, not a path to the record itself. The renderer introduced below preserves that computation and its output. This distinction prevents a public head query from being relabeled as a snapshot merely because its computation used private bytes.

Definition 4.2 (Record acquisition and readability)**.** A successful $\mathsf{Capture}(h)$ returns the actual retained bytes of h and adds an immutable copy to the adversary's archive $\mathcal{A}_t$. Let $\mathcal{K}_t$ be the wrapping-key identities disclosed by time t. Define

$$\mathcal{C}_t = \{v : \exists h \in \mathcal{A}_t,\ \exists \gamma,\ (v, \gamma) \in \mathsf{paths}(h)\}, \tag{4}$$

$$\mathcal{L}_t = \{v : \exists h \in \mathcal{A}_t,\ \exists \gamma,\ (v, \gamma) \in \mathsf{paths}(h)\ \wedge\ \gamma \subseteq \mathcal{K}_t\}. \tag{5}$$

Each capture records $\mathsf{AcquireOriginal}(v, t)$ for every version in its support, including versions already acquired through another path. An unsuccessful request for unavailable state returns its ordinary error and adds no acquisition.

Both sets describe the specified disclosure interfaces, using provenance and key identities. They do not test whether the adversary has guessed a secret. The read set can grow without another capture: if h was obtained at t_1 and its final missing wrapping key is disclosed at $t_2 > t_1$, the corresponding versions enter $\mathcal{L}_{t_2}$. Host deletion at an intermediate time leaves h in $\mathcal{A}_{t_2}$. Acquisition, by contrast, already occurred at t_1.

An additional key that is explicitly recoverable from an opened record can be included by a declared extraction rule. In this extension, iterate (5) and the declared key-extraction rules to their least fixed point after each event. Each rule identifies its input bytes, its deterministic extraction algorithm, and the resulting actual key identity. Only rules with available inputs fire. This accounts for nested protection without assuming that inaccessible keys have already been learned.

Signer-file capture is a separately typed disclosure. For correspondence with the retained symbolic theorem, it records exposure of the actual signing key conservatively at acquisition, as does that theorem's signer-file rule. The readability ledger can additionally record when the file's contents become decryptable. In particular, a wrapping-key disclosure cannot be scoped to traffic state when the declared capture also includes the matching signer file.

Constructive exposure. From public protocol inputs, explicit leaks, and readable record payloads, we form a labelled replay-dependency graph. Its edges are specified algorithms: unseal a reservation with its matching key; regenerate a key from its original tape; replay encapsulation on the bound offer; decapsulate a published ciphertext; derive a root; and derive the direction-specific traffic and ACK material. Directly stored roots, skipped keys, and plaintext enter through their own record fields. For the implemented root schedule, an edge is

$$(R_{e-1}, S_e, C_e) \longmapsto R_e = \mathsf{HKDF}(R_{e-1}, S_e, D_{\mathrm{root}} \| H_{\mathrm{core}}(C_e), 32).$$

Closing these explicit edges over the nodes of the bounded execution yields a computable set of message keys with replay witnesses. It supplies positive exposure certificates; the confidentiality game below is the separate criterion for messages outside the declared disclosure conditions. Confirmation counters belong to the public progress state and do not reset this graph.

4.3 An original-record implementation of the copy interface

We now isolate a property of record management that is independent of KEM secrecy. The following two storage machines use the same protocol machine $\mathcal{P}$, random-tape source, scheduler, and cryptographic interfaces.

The *literal-copy machine* materializes the immutable payload X_v in each indicated serialization position. Its objects include current images, pending targets, and retained historical copies. It records the origin tags alongside these values as private bookkeeping.

The *original-record machine* keeps one immutable table entry $\mathcal{O}[v] = X_v$ and represents each serialization position by a reference to v. Its wrapper maintains the physical-object directory, pending states, adversary archive, origin tags, and key identities. It has the following specified interfaces.

1. $\mathsf{RegisterOriginal}(v, X_v)$ is a private write by the honest machine. It fixes one value at a fresh v. The wrapper receives the handle and the provenance metadata.
2. $\mathsf{RenderOriginal}(\theta)$ evaluates the supplied serialization recipe θ by dereferencing $\mathcal{O}$. A recipe specifies literal public fields, record references, references to already rendered objects, and each sealing, hash, MAC, or other admitted byte-processing call. A fresh sealing call uses a fresh invocation index and the same prescribed coin distribution as the literal-copy machine. An exact-object read or replay returns its cached original bytes.
3. $\mathsf{AcquireOriginal}(v, t)$ records acquisition of that version. This is an event, not an assertion of decryption. A capture's externally returned bytes come from the rendered object cache.
4. $\mathsf{RevealOriginal}(v)$ returns the original payload X_v. The wrapper uses this interface for a plaintext record-opening response only after a captured path is enabled by the disclosed keys, and caches the answer for subsequent openings.

The renderer is an explicit trusted interface in this intermediate experiment. It has access to the original payload table and evaluates the stated byte operations, including operations on undisclosed payloads. Consequently, it can produce real ciphertexts, commitment digests, and report MACs. This definition does not supply an algorithm for producing those bytes from an unknown KEM-challenge tape. It gives a precise location for the subsequent cryptographic replacement of that interface.

The operational contract has three further requirements. First, all publicly visible payload-dependent bytes and statuses are obtained through the specified renderer or through $\mathcal{P}$'s common protocol computation. Memory addresses and host timing are not additional outputs of this interface. Second, actual byte-operation order, error responses, partial transaction outcomes, and cryptographic-oracle calls are retained. Caching an exact object does not discard a native hash-validation call, observable retry, or write attempt: such an action remains a renderer or protocol action. Third, the reference metadata and the key-identity relation are available to the wrapper without inspecting a selected hidden reservation's coins. These are checkable interface conditions; none asserts the secrecy of a record, a root, or a message.

Lemma 4.3 (Alias normalization)**.** *For the operations and renderer contract above, every adversary $\mathcal{A}$ interacting with the literal-copy machine has an online wrapper $\mathcal{B}$ interacting with the original-record machine such that their public views have the same distribution. Their reservation-origin ledgers, record-save events, acquisition events, and read sets are identical after projection onto the original handles.*

The wrapper makes at most $|\mathcal{L}_T|$ distinct $\mathsf{RevealOriginal}$ *calls by terminal time T. Repeated physical copies, captures, and openings do not multiply this number. The wrapper preserves every original reservation and every fresh sealing invocation, so it does not merge their respective cryptographic resource counts.*

The statement also holds for an adversary with quantum private state and quantum access to common public cryptographic oracles, provided the storage interface is classical and both experiments retain the same oracle calls.

Proof. Maintain a map from every literal record occurrence to its immutable handle v, and from every native object identifier to a rendered-object entry. Couple the protocol

random tapes, sealing coins, scheduler choices, and public cryptographic oracles between the experiments. The invariant is that dereferencing every original-record recipe produces the corresponding literal-copy bytes, with the same semantic provenance and physical disposition.

For a new record, register its payload once and extend the occurrence map. For a fresh rendering, submit the same recipe and byte-operation sequence to RenderOriginal. Dereferencing gives the same inputs to every sealing, digest, MAC, validation, and output operation. Coupled coins and common oracles therefore give the same bytes, statuses, and state changes. This step retains a write that became durable before returning an error; the caller's status does not overwrite the persistence state.

An exact copy or pending-target application copies the object reference and its original cache entry. Aggregation collects the corresponding original references, and resealing evaluates the new recipe using those references. Each operation preserves the session-record handles and their save times. It follows by induction on the recipe constructors that $\mathsf{paths}(h)$ is the same in both machines. A public hash or MAC constructor contributes its common output, rather than a spurious record-acquisition path.

On successful capture, output the cached object bytes in both experiments and append its immutable descriptor to the adversary archive. For each version in its support, emit the same acquisition event at the actual capture time. Preserve the ordinary error for an unavailable object. When a matching wrapping key is disclosed, use the common key identity and reevaluate the enabled paths in the complete adversary archive. For an opening response, obtain X_v from RevealOriginal on its first use and from the wrapper's cache thereafter. Every such call has $v \in \mathcal{L}_T$. Shared-key aliases and later key acquisition are thus handled without another copy of X_v. The optional key-extraction extension preserves the invariant by induction on the least-fixed-point rounds.

Host deletion removes only host references. The adversary archive and already returned payloads persist. Crash and resume use the same retained object identifiers; neither operation recreates a reservation or changes a save event. Ordinary protocol actions are performed by the same $\mathcal{P}$ with the same results. This completes the induction, including adaptive choices based on any preceding response.

For quantum private state, identical classical responses and the same public oracle transformations induce the same transformation of that state at each interaction. The argument introduces no measurement of a quantum oracle query and no oracle-dependent advice. Finally, caching each plaintext opening by v proves the query bound; reservation and fresh-seal indices were preserved separately throughout. □

The lemma places multiplicity in the correct ledger. An adversary may retain many ciphertext aliases of one version, and those ciphertexts may have many different public heads. All such byte-level observations remain in the view. The repeated payload uses one original-record opening, whereas each genuinely fresh seal and each public hash or MAC computation keeps its own resource accounting.

4.4 A recovery experiment with acquisition-based eligibility

Definition 4.4 (Fixed-target recovery experiment)**.** Fix a protocol profile $\mathcal{P}$ and finite execution budgets, together with a designated session s, recovery cut c, target epoch e, and candidate generation and encapsulation reservation handles g_e, a_e. Handles may be named by original invocation slots before those invocations occur; a missing invocation will fail the reservation-event test below. Public protocol messages may be delayed, duplicated, reordered, or replaced. The adversary can capture the declared retained objects, reveal wrapping keys, reveal actual signing keys or signer files, and reveal designated reservation tapes. The cut

exposes the selected wrapping keys; preceding roots may already be public or disclosed by the stated prior captures. These disclosures have their exact scope in the event ledger.

For one fresh logical application send, the adversary supplies equal-length messages (m_0, m_1). The challenger samples $b \leftarrow \{0,1\}$ and runs the ordinary sending transition on m_b. Rejection or an unknown outcome keeps its ordinary protocol behavior. A retained accepted send is identified by (s, e, d, j), including direction d and the epoch-scoped logical index j; exact retries reuse its original ciphertext. The challenge is valid only when that logical send is accepted in the designated epoch. Subsequent protocol actions and disclosures continue until the adversary outputs b'.

Write ι_A, ι_B for the two actual signing-key identities configured for the target session. The predicate $\mathsf{ReservedFor}(s, e, g_e, a_e)$ checks the occurrence of the named honest generation and encapsulation reservation events in the target epoch's local durable plans, with their target session and roles. It checks event metadata, without asserting that their outputs form the accepted target exchange. Eligibility uses the following event conditions over the complete execution:

$$\begin{aligned}\mathsf{Eligible}_{s,e,c}(\tau) \iff\ & \mathsf{ValidTarget}(\tau) \ \wedge\ \mathsf{ReservedFor}(s, e, g_e, a_e) \\ & \wedge\ c < \mathsf{res}(g_e) \ \wedge\ c < \mathsf{res}(a_e) \\ & \wedge\ \iota_A, \iota_B \notin \mathsf{IdentityExposed}(\tau) \\ & \wedge\ \mathsf{EntropyExposed}(s, e; \tau) = \varnothing \\ & \wedge\ \nexists v \in \mathcal{C}_T : \mathsf{subject}(v) = s \ \wedge\ c < \mathsf{save}(v). \end{aligned} \tag{6}$$

Here $\mathsf{EntropyExposed}(s, e; \tau)$ records the target epoch's designated entropy disclosures. The separate event Attr states that these named reservation events form the accepted target's unique usable generation/encapsulation pair, with an adapter-computable association to the original slots. In particular, it connects the generation public key, the encapsulation input, the authenticated response, and the target's subsequent key schedule. This matching is a source/authentication obligation, rather than an additional eligibility filter: a failure of Attr can occur while $\mathsf{Eligible}_{s,e,c}$ holds and is charged in the computational accounting.

The final line of (6) concerns every saved version of session s, across its roles and epochs. It uses acquisition $\mathcal{C}_T$, including protected bytes, rather than delayed readability $\mathcal{L}_T$. Other epochs' entropy disclosures retain their ordinary behavior. Both peers' confirmation is not an additional send prerequisite.

Define the unconditional centered advantage by

$$\mathsf{Adv}^{\mathrm{rec}}_{\mathcal{P}}(\mathcal{A}) = \left| \mathbb{E}\left[\mathbf{1}_{\mathsf{Eligible}_{s,e,c}(\tau)} (-1)^{b \oplus b'} \right] \right|. \tag{7}$$

Invalid or ineligible executions contribute zero. This definition retains failures and adaptive disclosures in the probability space and avoids conditioning a reduction on an uncontrolled eligibility probability. If every execution with $\mathsf{Eligible}_{s,e,c} = 0$ is completed by an independent fair guess, the conventional advantage over one half is

$$\mathsf{Adv}_{\mathrm{msg}} = \left| \Pr[\text{completed guess is correct}] - \tfrac{1}{2} \right| = \tfrac{1}{2}\, \mathsf{Adv}^{\mathrm{rec}}_{\mathcal{P}}(\mathcal{A}).$$

The computational accounting uses this latter convention.

The adaptive version retains one application challenge and permits the candidate selector to choose its coordinates and reservation-event handles from the observed interaction, rather than fixing them at setup. Eligibility requires those coordinates to identify the challenge's accepted logical send; selection never transfers its test bit to another send. Eligibility and the usable-pair attribution are evaluated on that experiment's complete history. An origin-pair

reduction fixes a slot pair in $[G] \times [C]$ and scores the event that the final selector attributes the target to that pair. The selector and disclosure predicates are recomputed in each comparison world. No independence between that selection and the transcript is imposed.

The record restriction in (6) is the one used by the retained multi-epoch symbolic theorem: its Snapshot action outputs a protected record, while Cut separately releases the wrapping keys. The theorem establishes symbolic plaintext secrecy with both original reservations after the cut, key-scoped identity exclusions, the target entropy exclusion, and the stated post-cut saved-record acquisition exclusion [Q-P26]. Its joint replay verifies the main lemma and all 24 reusable dependencies. Equation (7) specifies the computational message experiment to which the cryptographic modules are to be connected; the alias lemma establishes its record-interface normalization.

Corollary 4.5 (Safe stopping at a forbidden acquisition)**.** *Consider the fixed-target experiment of Definition 4.4. Suppose an actual successful capture contains a target-session version v with $c < \mathsf{save}(v)$, identified by the original-record metadata. For any terminal bit D, replacing the remainder of that execution by output zero leaves*

$$\Pr[\mathsf{Eligible}_{s,e,c}(\tau) \wedge D = 1]$$

unchanged. Replacing its terminal score by zero also leaves the expectation inside (7) *unchanged.*

In particular, if a captured target-session version retains the selected reservation $o \in \{g_e, a_e\}$, then $c < \mathsf{res}(o)$ and (3) *establish this stopping condition. The wrapping key may be obtained before or after the capture.*

Proof. The successful capture inserts v into $\mathcal{C}_t$ immediately. Its save time and subject are immutable, and acquisition remains recorded after host deletion, resealing, confirmation, or process loss. Thus the last conjunct of (6) is false for every continuation. Each such continuation already contributes zero to both quantities. On executions that never reach this condition, the experiment is unchanged. The final assertion follows from $c < \mathsf{res}(o) \leq \mathsf{save}(v)$. □

The stopping test uses the successful acquisition event and its record metadata; it can run before the response's private bytes are serialized by a challenge adapter. It is not an SDK rejection of the capture. A public digest, an unrelated record, or an unsuccessful capture keeps its real behavior. In particular, difficulty computing a permitted digest or report MAC does not trigger this corollary. For adaptive target selection, stopping is applied to the corresponding fixed-candidate event in the reduction. One possible candidate becoming ineligible does not justify terminating a game that can still select a different eligible target.

Two short executions illustrate why both kinds of origin are necessary. If v was saved before c and a sibling write reseals it after c, the acquisition still refers to the original pre-cut v. The new global image time alone does not falsify the final conjunct of (6). If instead v was saved after c, a later write attributed to a sibling does not change its subject or make its capture eligible. Likewise, a pre-cut generation reservation retained through a delayed receipt remains pre-cut when its KEM computation completes later. Its reservation condition, rather than the slower endpoint's confirmed counter, determines that case.

4.5 Connection to the cryptographic argument

Lemma 4.3 gives an exact structural result for the specified interface. Its renderer preserves the actual bytes and all cryptographic dependencies; it performs no KEM or hash replacement. Corollary 4.5 supplies a usable stopping rule for a fixed-target eligible-event reduction.

Together they remove copy multiplicity and delayed decryption from the logical ambiguity of that interface while keeping the acquisition predicate intact.

The next cryptographic connection is the implementation of RenderOriginal when a selected PQ reservation is an opaque KEM challenge handle. The required outputs are the actual permitted public heads, intent digests, report MACs, metadata, and error responses, with original-object caches and all shared-oracle preparation preserved. The existing commitment and cleanup-report modules address named subinterfaces of this connection. Successful forbidden acquisition can use the stopping corollary; public rendering and permitted acquisitions require their own consistent answers.

The native source correspondence has a concrete form: map every in-scope current image, pending target, and retained copy to the original version handles, their first durable save events, and their wrapping-key identities; show that the source transitions implement the constructor rules above. Arbitrary byte residues or partial captures require their actual projection interface, rather than automatic promotion to a whole-record snapshot. This correspondence also supplies the usable original-slot attribution Attr. It must be evaluable from the adapter's metadata, without testing an unknown challenge tape. Eligibility remains meaningful on executions where that attribution fails.

After that connection, the existing original-operation KEM adapter addresses one selected generation/encapsulation pair: exact challenge-ciphertext repeats share the same component result and one correctness event. For adaptive attribution the slots belong to the fixed set $[G] \times [C]$, with origin and eligibility evaluated in each comparison experiment. Copies of a reservation do not create more slots, while genuinely new draws do. Alias normalization supplies no additional multiplicative loss; the component reductions and adaptive-target accounting retain their own resource terms.

The subsequent root argument uses the actual order

$$p_e = \mathsf{HMAC}_{R_{e-1}}(S_e), \qquad R_e = \mathsf{HMAC}_{p_e}(D_{\mathrm{root}} \| H_{\mathrm{core}}(C_e) \| \mathtt{01}).$$

When R_{e-1} is disclosed, the recovered secret occupies the first HMAC's message input. The source-derived extraction modules address that call shape. Their contribution-replacement interface is before the root's confirmation tags: downstream signatures, transcript digests, traffic, ACK keys, and later epochs are recomputed consistently in each hybrid. Shared HMAC inputs and exact retries use consistent values. This supplies the precise division of work: the present result proves record-interface normalization and acquisition-based stopping; the accepted prefix and key-schedule composition establish computational recovery for the experiment in (7).

5 Established recovery result and separating executions

5.1 The multi-epoch symbolic theorem

The checked symbolic model includes inherited roots, alternating proposers, asymmetric confirmation, initial traffic, and unbounded epochs. Signing and wrapping keys may be shared across their modeled uses. Its Cut event exposes the selected wrapping keys; exposure of other epochs' entropy is represented separately. We state the main theorem in terms of its recorded events, preserving its original target and disclosure predicates.

Let τ be a trace of that model. Write $\mathsf{SessionKeys}(s, a, b)$ for the actual identity keys associated with session s, $\mathsf{KeyReserved}(s, e, P)$ and $\mathsf{EncapReserved}(s, e, Q)$ for the target reservations, and $\mathsf{Message}(s, e, r, k, m)$ for an application-message event. Define $\mathsf{AcquiredAfterSave}_c(s)$ to hold when there exist a role, an epoch, a revision, and matching Saved/Snapshot events for session s, with the *save* event strictly after c. The matching record's session projection is the relevant saved object.

Theorem 5.1 (Checked multi-epoch symbolic secrecy)**.** *Suppose τ contains the corresponding* SessionKeys, Cut *at c, target reservation events at g and h, and* $\mathsf{Message}(s, e, r, k, m)$*. If*

$$c < g, \qquad c < h,$$

neither actual target identity key is disclosed anywhere in τ, no reserved entropy of target epoch e is disclosed, and $\mathsf{AcquiredAfterSave}_c(s)$ *is false, then τ contains no adversarial knowledge event* $\mathsf{K}(m)$*.*

The theorem is the retained query `multi_epoch_fresh_reservations_recover_symbolic_message_secrecy`. Its certificate was replayed with all 24 reusable dependencies in the same Tamarin process. The independent 4 November replay used Tamarin 1.12.0 and Maude 3.5.1, one worker, a 12-GiB heap bound and the unchanged 900-second limit; it completed in 816.903 seconds. The output certificate was byte-identical to the retained input. Appendix A gives its digest and archive member, making the theorem's exact formula and proof available for inspection.

The complete inventory contains 40 queries: 27 verified and 13 incomplete. The latter comprise eleven multi-round existence queries and two historical all-trace queries outside the main theorem's dependency group. Two subsequent finite ground-trace constructions supply the concrete executions discussed below. Their independently checked formulas and original certificate inventory remain separately identifiable.

The theorem uses the modeled cryptographic constructors and record interface. Its key scope follows actual identity ownership: a shared signer exposed through another session is still a disclosure of that target key. Likewise, a retained target projection saved after Cut remains subject to the acquisition condition when a sibling write carries it into a later aggregate image.

5.2 Executability and the freshness boundary

A finite two-round construction exercises the operational rules with an exposed ancestor root and satisfies the complete original positive existential formula. A separate source-derived evaluator checks its rule instances and global quantifiers. This provides a concrete execution for the modeled recovery premises.

The delayed-receipt construction instead contains a pre-Cut epoch-two generation reservation and a post-Cut encapsulation. It derives the target payload while satisfying the remaining main-theorem antecedents. Its point audit identifies $c < g$ as the sole false antecedent. Seven countertraces are rejected by the witness checker. Together, the positive and separating traces connect the recovery condition to executable schedules.

5.3 Native reservation disclosure

The native experiment stops a child process after committing a reservation, kills and reaps it, reopens the journal, and inspects the retained phase. It exercises two distinct boundaries:

- a 277-byte key-generation token, before key computation or offer publication;
- a 245-byte encapsulation token, after offer authentication and before encapsulation computation or response publication.

The predictor obtains the previous root, journal identity, wrapping key and captured token. Thereafter it uses the public policy configuration and later authenticated control wires. Its prediction interface has no journal or filesystem input, and the experiment withholds signer files and future private owners. A wrong wrapping key is rejected.

For each cut, both honest peers complete the rekey and reopen their journals. The driver then creates fresh 128-byte application payloads, sends three in each direction, and checks honest delivery. Independently reconstructed traffic owners authenticate and decrypt all six intercepted frames. The two cuts therefore yield twelve actual post-restart plaintext recoveries. The delayed-receipt extension repeats the six-message check at epoch two using one responder snapshot.

Proposition 5.2 (Execution-time and counter-based separation)**.** *In the retained native executions, computing a KEM after restart, completing the target epoch, and subsequently reopening the journal do not make a previously disclosed reservation an independent secret contribution. Moreover,* $\min(c_A, c_B) + 1$ *is not a universal upper bound on exposed epochs.*

Proof. At both process-cut boundaries the captured reservation reconstructs the later honest KEM result exactly. Equations (1)–(2) then reconstruct traffic keys from the captured predecessor root and public wires; authenticated decryption verifies the six corresponding plaintexts after completion and restart. In the delayed-receipt execution, $(c_A, c_B) = (0, 1)$ at capture and six epoch-two plaintexts are recovered. Thus the proposed counter bound equals one while an exposed epoch equals two. □

5.4 An aggregate write preserves the target origin

A separate two-session native execution changes a sibling session and writes a new aggregate image. It checks exact equality of the retained target record, reopens and retries the original operation, and decrypts six subsequent target messages in each build profile. The new global revision and outer encryption nonce coexist with the old target reservation. This execution instantiates the alias/provenance distinction in the record model.

All native results above are claims about their identified executions. The symbolic theorem supplies the universal statement within its model; the computational accounting in Section 7 specifies the next connection between the concrete record format and message security.

6 Executable reference corpora and comparison design

The executable references provide reproducible observations of communication and exposure. The pinned SPQR source is `f2589fef855c10f39d72634dab3d14654dd410bf`. Seven schedules each contain 2,048 sends: alternating, loss, reordering, duplication, 9:1 asymmetry, offline-then-exchange, and one-way traffic. Six capture points for each endpoint produce 84 passive snapshot cases. The whole-KEM control runs those schedules at local-send intervals 1, 32 and 64, producing 252 corresponding cases. Together these are 28 configuration/schedule combinations and 336 snapshots. The explicitly modified 64-byte SPQR variant has its own source identity.

6.1 Observed derivation and communication

The snapshot predictor receives one endpoint's serialized state and the full public transcript, including packets lost before peer delivery. A separate harness compares every derived key with the actual sender's key. Public deterministic seeds are test fixtures; the measurement establishes causal key derivation through the declared predictor interface. The output label `not_derived` denotes that predictor's result. It is separate from the security predicate of a computational experiment.

Table 2 reproduces the retained alternating-schedule observations. The snapshot is taken at A immediately after its first send, leaving 2,047 future sends. The pending KEM key

Component/profile	Mean bytes/send	Peak frame	Epochs A/B	Derived keys
Pinned SPQR, 32 B	32.150	37	24/23	172
SPQR variant, 64 B	58.971	69	46/45	88
Whole KEM, interval 1	819.658	1,225	683/683	4
Whole KEM, interval 32	44.047	1,223	32/32	66
Whole KEM, interval 64	25.023	1,223	16/16	130

Table 2: Retained component-level alternating workload. Byte counts are the component wire contribution; epoch counts are each component's root/output observations. The constructions have different authentication and retention graphs. Outer application AEAD, transport, durable storage and energy are outside these measurements.

accounts for 87 of the 172 derived keys in the pinned 32-byte SPQR case, demonstrating why the currently emitted chain epoch alone does not describe exposure.

The one-way schedule produces no fresh PQ epoch in these references and the predictor derives every post-snapshot key. The whole-KEM active-control experiment addresses a different property: one initial endpoint-state capture allows two attacker forks to replace subsequent exchanges while honest confirmed epochs increase. All 1,536 sender keys across three 512-send profiles match the attacker-derived keys. This supplies a concrete distinction between fresh KEM execution and authenticated recovery when the ongoing authentication authority is itself derived from captured session state.

6.2 Aligning the Continuity experiment

A matched experiment must give each construction the same application and control-message opportunities and the same loss, reorder and duplication schedule. Authentication capabilities are recorded explicitly: the whole-KEM control uses session-state authentication, whereas Continuity controls use the separately modeled device signing authority. Serialized volatile state, retained reservations and actual durable journal images are distinct disclosure objects.

The planned Continuity comparison records, for each exposure case, the original generation and encapsulation reservation identifiers, the retained record projection, acquired wrapping-key identifiers, endpoint phase/counter state, public-wire availability and the actually reconstructed message set. Communication records include total bytes and largest frame. Persistence records include durable writes, restart/retry work and retained-state size. Each timing denominator covers its declared operation interval and names the compiler profile, primitive backend, host and sample unit.

The principal ablations hold the underlying reservation fixed while changing its execution time, delay a receipt, copy a target through a sibling aggregate write, and compare origin-preserving labels with labels based on the last image write. These ablations test the mechanisms isolated by the theorem and separating executions. Their comparison will explain both exposure and cost under each matched workload.

7 Computational accounting and the common experiment

The origin model provides the accounting boundary for a computational experiment. Each original generation or encapsulation invocation receives a slot when its entropy is reserved, including attempts that later fail. Replaying or copying that reservation retains its slot. Fix public lifetime budgets $G, C \geq 1$ and write $J = [G] \times [C]$. Unused slots remain unused. These proof-resource budgets differ from a bound on the number of records resident at one time.

7.1 A selected-target experiment

Let V be the virtual original-record experiment with the real KEM component. The adversary chooses its target after its observed interaction. Eligibility E retains the selected recovery conditions: both original target reservations follow Cut, the actual target signing keys and target-epoch entropy remain unexposed, and no selected-session record saved after Cut is acquired. The experiment follows allowed asynchronous control and application transitions, including sending after the local FinalReady transition.

Let Attr mean that the selected target has a unique usable pair of original slots, and let $I \in J$ denote that pair. The mapping and every event used by the reduction must be computable from the adapter's known semantic state and observations. Let

$$\eta \;\geq\; \Pr_{V}[E \wedge \neg\mathsf{Attr}].$$

For $j \in J$, define V_j as the full experiment with the adapter's random component override at pair j. Its transcript, eligibility and target choice are recomputed in that world. The remaining protocol and oracle calls use the same common interfaces.

The application test bit is sampled by the surrounding simulator. Write $S = +1$ for a correct guess and $S = -1$ otherwise; invalid tests contribute zero through E. Every $E = 0$ execution is completed by an independent fair guess. Thus the usual message-guessing advantage is $\frac{1}{2}|\mathbb{E}[ES]|$. The simulator can evaluate this score without observing the hidden bit of its component KEM challenger.

7.2 Centered-score composition

For each j, put $E_j = E \wedge \mathsf{Attr} \wedge (I = j)$. Suppose the fixed-pair component adapter gives, for $\sigma \in \{+1, -1\}$,

$$\left|\Pr_{V}[E_j \wedge S = \sigma] - \Pr_{V_j}[E_j \wedge S = \sigma]\right| \leq a_j^{\sigma} + \delta_j, \tag{8}$$

where a_j^{σ} is the advantage of the actual derived component distinguisher and δ_j is the corresponding correctness term. The distinguisher includes target selection and all relevant oracle queries. Define

$$b_j = \mathbb{E}_{V_j}[E_j S].$$

Finally, let ϵ_{rec} bound the binary distinguishing gap between the native and virtual experiments for their efficiently evaluable positive-score and negative-score events. This is the concrete record-interface connection, including public outputs and accepted operation histories.

Proposition 7.1 (Adaptive origin-pair accounting)**.** *Under these stated interface and fixed-pair bounds, the native application's guessing advantage satisfies*

$$\mathrm{Adv}_{\mathrm{msg}} \leq \min\left\{\frac{1}{2},\; \epsilon_{\mathrm{rec}} + \frac{1}{2}\left(\eta + \sum_{j\in J}|b_j| + \sum_{j\in J}(a_j^{+} + a_j^{-} + 2\delta_j)\right)\right\}. \tag{9}$$

Proof. In the single experiment V, the attributed events E_j are disjoint. Decomposing its score yields

$$\mathbb{E}_V[ES] = \mathbb{E}_V[ES\,\mathbf{1}_{\neg\mathsf{Attr}}] + \sum_{j\in J}\mathbb{E}_V[E_j S].$$

The first term has magnitude at most η. Each attributed expectation is the positive-event probability minus the negative-event probability. Applying (8) to those two events and then the triangle inequality bounds its distance from b_j by $a_j^{+} + a_j^{-} + 2\delta_j$. Summing gives the virtual score bound. The two native/virtual binary-event comparisons contribute at most $2\epsilon_{\mathrm{rec}}$ to the score difference. Dividing by two and using the maximum bit-guessing advantage proves (9). □

The selector may depend adaptively on its entire observed transcript. The absolute values in $\sum_j |b_j|$ retain each override world's own target selection. In particular, the override probabilities are not normalized as though they belonged to one common ideal distribution. The score decomposition also retains failed and ineligible executions in the experiment, avoiding an uncontrolled division by the probability of eligibility.

Uniform bounds $|b_j| \le \beta$, $a_j^\sigma \le a$, and $\delta_j \le \delta$ give

$$\mathrm{Adv}_{\mathrm{msg}} \le \min\left\{\frac{1}{2},\ \epsilon_{\mathrm{rec}} + \frac{\eta}{2} + \frac{GC}{2}(\beta + 2a + 2\delta)\right\}.$$

Alias proliferation changes the record inventory while preserving the original slot count. This is the point at which origin-preserving record accounting enters the reduction.

7.3 The joint root and message interface

The root derivation in (1) begins with

$$p_e = \mathrm{HMAC}_{R_{e-1}}(S_e).$$

After disclosure of R_{e-1}, the secret is in HMAC's message input. The applicable extractor/swap-PRF condition therefore concerns that input distribution; ordinary PRF security for an unknown HMAC key is a different premise [BBGS23, KE10]. The retained computational notes instantiate the actual input lengths and domains, treat shared root uses and aliases, and state the corresponding quantum-oracle accounting.

The quantities in (9) organize the next two cryptographic connections. To bound ϵ_{rec} and η, the concrete adapter must simulate protected pending writes, public commitments, replay results, report keys and permitted acquisition using original operation handles, while matching the source's accepted histories. To bound each b_j, the random-component world must jointly generate root extraction, confirmation tags, traffic, acknowledgements and successor epochs. Replacing a contribution before producing its dependent controls keeps those descendants consistent.

The current record-interface lemma and source-derived event definitions isolate the provenance part of the first connection. The existing root-extraction, shared-HMAC, KEM-adapter and adaptive-target modules supply its cryptographic interfaces and the second connection's component statements. Equation (9) is a proved accounting proposition with these explicit inputs; completing the common experiment requires concrete bounds for those inputs.

References

[ADJ+25] Benedikt Auerbach, Yevgeniy Dodis, Daniel Jost, Shuichi Katsumata, and Rolfe Schmidt. How to compare bandwidth constrained two-party secure messaging protocols: A quest for a more efficient and secure post-quantum protocol. In *34th USENIX Security Symposium (USENIX Security 25)*, pages 6717–6736. USENIX Association, August 2025.

[BBC+26] Karthikeyan Bhargavan, Maxime Buyse, Graeme Connell, Franziskus Kiefer, and Rolfe Schmidt. A formal analysis of Signal's sparse post-quantum ratchet protocol and its implementation. In *Proceedings of the 2026 ACM SIGSAC Conference on Computer and Communications Security*. ACM, 2026. To appear; listed in the official conference program.

[BBGS23] Matilda Backendal, Mihir Bellare, Felix Günther, and Matteo Scarlata. When messages are keys: Is HMAC a dual-PRF? In *Advances in Cryptology—CRYPTO 2023*, volume 14083 of *Lecture Notes in Computer Science*, pages 661–693. Springer, 2023.

[Bha25] Karthikeyan Bhargavan. Helping secure Signal's post-quantum transition. Cryspen, October 2025. October 2, 2025.

[CGCD+20] Katriel Cohn-Gordon, Cas Cremers, Benjamin Dowling, Luke Garratt, and Douglas Stebila. A formal security analysis of the Signal messaging protocol. *Journal of Cryptology*, 33(4):1914–1983, 2020.

[CJN23] Cas Cremers, Charlie Jacomme, and Aurora Naska. Formal analysis of session-handling in secure messaging: Lifting security from sessions to conversations. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 1235–1252. USENIX Association, 2023.

[CJR26] Vincent Cheval, Charlie Jacomme, and Jessica Richards. Automated formal analysis of Signal's double ratchet: Attacks, fixes and security proofs. In *2026 IEEE Symposium on Security and Privacy (SP)*, pages 1522–1538. IEEE, 2026.

[CMN25] Cas Cremers, Niklas Medinger, and Aurora Naska. Impossibility results for post-compromise security in real-world communication systems. In *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2025.

[DJK+25] Yevgeniy Dodis, Daniel Jost, Shuichi Katsumata, Thomas Prest, and Rolfe Schmidt. Triple ratchet: A bandwidth efficient hybrid-secure Signal protocol. In *Advances in Cryptology—EUROCRYPT 2025*, volume 15608 of *Lecture Notes in Computer Science*, pages 302–331. Springer, 2025. Extended version, Cryptology ePrint Archive, Paper 2025/078.

[FGJ24] Marc Fischlin, Felix Günther, and Christian Janson. Robust channels: Handling unreliable networks in the record layers of QUIC and DTLS 1.3. *Journal of Cryptology*, 37, 2024. Article 9.

[HJKL26] Clément Hérouard, Charlie Jacomme, Adrien Koutsos, and Joseph Lallemand. Secrecy in Squirrel and the post-compromise security of a ratchet. In *Proceedings of the 2026 ACM SIGSAC Conference on Computer and Communications Security*. ACM, 2026. To appear; full version in Cryptology ePrint Archive, Paper 2026/1912.

[KE10] Hugo Krawczyk and Pasi Eronen. HMAC-based extract-and-expand key derivation function (HKDF). RFC 5869, Internet Engineering Task Force, May 2010.

[PMS25] Trevor Perrin, Moxie Marlinspike, and Rolfe Schmidt. The double ratchet algorithm. Signal specification, November 2025. Revision 4, November 4, 2025.

[Q-P26] Q-Periapt project. Independent joint replay of multi-epoch symbolic message secrecy. Q-Periapt research artifact, November 2026. November 4 replay; fixed repository checkpoint 228c944fc476e3e57d6c08063cacb19bca5b360e.

A Source and evidence map

The source snapshot for this manuscript's established protocol and experiment records is commit `228c944fc476e3e57d6c08063cacb19bca5b360e` of `billlza/q-periapt`. Each retained execution keeps its original source and tool identity. The local record model and accounting proofs in this manuscript are mathematical arguments; the following entries identify the independently recorded mechanization and native observations.

Result	Reproduction source
Joint symbolic replay	20261104-multi-epoch-replay. The original archive is `CAPTURE.zip`; the checked certificate member is `replay-01/01-baseline-replay.proof.spthy`.
Positive two-round trace	20261105-two-round-ground-trace.
Delayed-receipt ground trace	20261105-delayed-receipt-ground-trace.
Native reservation disclosure	RESERVATION_DISCLOSURE.md and `src/durable/messages/tests/reservation_disclosure.rs` in the identity candidate.
Aggregate-copy execution	20261105-aggregate-reservation-copy.
Root, traffic and phase mapping	RECOVERY_DEPENDENCIES_V1.md; native `rekey/response.rs` and `rekey/completion.rs`.
SPQR snapshot corpus	continuity-spqr-reference: `TRACE_CORPUS.json`, `COMPROMISE_CORPUS.json`, and `COMPROMISE_EXPERIMENT.md`.
Whole-KEM corpus	continuity-whole-kem-reference: `TRACE_CORPUS.json`, `README.md`, and `ACTIVE_COMPROMISE.md`.
Computational component arguments	COMPUTATIONAL_RECOVERY_V1.md and its linked KEM adapter, hidden-record, shared-HMAC, root-extraction and adaptive-target notes.

The jointly checked certificate has SHA-256

`dde5d06a071b1560da0d4c28c3f1fd81a3c39708f9fd8564ee391ddc362bcf7f`.

Its source-base receipt identifies `6084f3bfb09333f6b66291b38809ed512296a27b`. The fixed 4 November replay retained all 40 queries and its exact tool, resource and output receipts. Reading and checking this retained artifact while preparing the manuscript is distinct from executing another Tamarin run.